CS 305ES Plantilla para el Proyecto final I

Historial de revisión de documentos

Versión	Fecha	Autor	Comentarios
1.0	18/09/2026	Julian Tovar	

Cliente



Instrucciones

Presente este informe de evaluación de la vulnerabilidad debidamente completado. Reemplace el texto entre corchetes con la información relevante. En este informe, identifique sus hallazgos de vulnerabilidad de seguridad y recomiende los siguientes pasos para remediar los problemas que ha encontrado.

- Responda a los cinco pasos descritos a continuación e incluya sus conclusiones.
- Responda con sus propias palabras. También puede incluir imágenes o material de apoyo. Si los incluye, asegúrese de agregarlos en los lugares pertinentes del documento.
- Consulte el documento Proyecto final I: Lineamientos y rúbrica para obtener instrucciones más detalladas sobre cada sección de la plantilla.

Desarrollador
Julian David Tovar Gaitan

1. Interpretar las necesidades del cliente

Artemis Financial tiene bajo su control información financiera de sus clientes relacionada con ahorro, jubilación, inversiones y seguros. En consecuencia, la confidencialidad, integridad y disponibilidad del contenido manejado constituyen necesidades básicas. Las comunicaciones entre los clientes, la API REST y los servicios internos deben estar protegidas para que no sea posible interceptar/modificar/reutilizar información sensible. La aplicación debe valerse de comunicaciones cifradas (HTTPS/TLS) y debe guardar las credenciales y otros secretos lejos del código fuente.

En el escenario, se indica que Artemis Financial desarrolla servicios para clientes localizados o bien en diversas partes del mundo, por lo que es razonable pensar que debe existir comunicación o procesamiento de información internacional. Esto implica que la empresa debe ocuparse de las regulaciones aplicables a protección de datos, privacidad y comunicaciones seguras de los países donde opere. Y también tiene que mantener mecanismos para actualizar las medidas de seguridad a medida que cambien las regulaciones.

En la lista de amenazas externas aplicadas se pueden observar la interceptación de la comunicación, el acceso no autorizado a información financiera, todo tipo de ataque de inyección, la explotación de vulnerabilidades en bibliotecas de terceros, la denegación de servicio y la explotación de configuraciones inseguras. Puesto que la aplicación en cuestión interactúa mediante una API REST, los parámetros que llegan desde los clientes tienen que ser validos antes de procesarlos.

La modernización también exige controlar las dependencias del código abierto. El proyecto se apoya en Spring Boot 2.2.4.RELEASE más varias bibliotecas asociadas, además de Bouncy Castle 1.46. Spring Boot 2.2.4 trabaja con bibliotecas de versión anterior, por ejemplo con Spring Framework 5.2.3.RELEASE. Las bibliotecas del código abierto pueden añadir funcionalidad importante, pero también pueden incluir vulnerabilidades conocidas si no se mantienen actualizadas. Por tanto, Artemis Financial debería establecer un proceso continuo de revisión de las dependencias y pruebas de seguridad.

2. Áreas de seguridad

APIs: La aplicación presenta unos endpoints REST utilizando el GreetingController y el CRUDController. En la medida que estos endpoints reciben solicitudes externas, deben implementar su correspondiente autenticación, su correspondiente autorización712, su correspondiente validación de parámetros713 y los correspondientes controles requeridos para los métodos HTTP permitidos.

Validación de entradas: el GreetingController recibe el parámetro name y el CRUDController el business_name. Los valores llegan desde solicitudes HTTP pero actualmente no se encuentra una validación explícita para ambos casos. La validación es necesaria para evitar que datos inesperados alcancen capas posteriores de la aplicación.

Criptografía: Artemis Financial maneja información financiera, por lo que la información que se transmite tiene que estar también protegida mediante TLS. También se deben proteger credenciales y cualquier otra información delicada que se almacene la cual el proyecto contiene

una dependencia muy antigua de Bouncy Castle (1.46), lo que incrementa el riesgo asociado a la utilización de componentes criptográficos viejos.

Código cliente/servidor: la aplicación se comporta como un servicio web y recibe solicitudes de clientes externos, por lo que el código en servidor debe asumir que todas las entradas externas pueden haber sido manipuladas y debe implementar las medidas de seguridad adecuadas antes de poder ser procesadas.

Manejo de errores/ calidad del código: En caso de que ocurra un error de conexión en el código de DocData, se lanzará una excepción mediante e.printStackTrace(). Las excepciones se mostrarán como un resultado por la salida estándar que puede mostrar información de la estructura interna de la aplicación, y también existen métodos no completos como por ejemplo en el caso de la clase myDateTime, lo que puede indicar que el código de la aplicación necesita ser revisado antes de ser usado en un entorno de producción.
Encapsulación: las clases deben proteger los datos y dar acceso limitado a datos sensibles. La clase customer por ejemplo, contiene datos de la cuenta, así como un método showInfo() que devuelve el número de la cuenta sin que exista un mecanismo a la vista que pueda estar relacionado con mecanismos de autenticación o autorización.
Estructuras de datos seguras: los datos financieros y las operaciones sobre cuentas deben utilizar estructuras y reglas que eviten que tomen valores no válidos. El método deposit() permite modificar la información del saldo sin comprobar que el valor que se ha recibido como argumento sea un valor válido, por ejemplo, depósitos negativos.

3. Revisión manual

Se hallaron los siguientes defectos evidentes tras una inspección manual del código fuente:

3.1. Credenciales directamente en el código de conexión de base de datos – DocData.java

La clase DocData tiene la siguiente conexión de base datos:

jdbc:mysql://localhost:3306/test","root","root"

Significa que en el código fuente se almacenan directamente la cuenta de usuario y contraseña de la base de datos, lo que si se expusiese el repositorio, se expondrían también las correspondientes credenciales.

3.2. Excepciones inseguras – DocData.java

El método read_document () lanza e.printStackTrace() en el caso de excepciones SQL. En una aplicación de producción, la manera de mostrar la traza de pila (stack trace) puede revelar información del funcionamiento interno de la aplicación y su entorno. Los errores tienen que ser registrados desde un mecanismo controlado, presentando al usuario mensajes genéricos.

3.3. Recursos de conexión de base de datos incorrectamente manejados – DocData.java

El objeto Connection se crea dentro del método pero no se cierra explícitamente. Esto puede llevar al agotamiento de conexiones u otros recursos si se invoca repetidamente el método. La conexión debería ser manejada por try-with-resources.

3.4. Inexistencia de autentificación y autorización en los endpoints – GreetingController.java y CRUDController.java

Los endpoints /greeting y /read no presentan ninguna forma de autentificación o autorización. Esto es de especial relevancia para una aplicación financiera puesto que las operaciones y los datos sólo deberían ser accesibles por usuarios autorizados.

3.5. Ausencia de validación explícita de entradas – GreetingController.java

En el método greeting() se recibe el name directamente mediante @RequestParam. Pese a que Spring realiza un cierto tipo de conversiones de tipos, el código no incluye ninguna restricción sobre longitud, contenido o formato del parámetro.

3.6. Ausencia de validación explícita de entradas – CRUDController.java

El endpoint /read recibe el business_name directamente:
@RequestParam(value="business_name") String name
No hay validación visible de longitud, formato, o contenido. Las entradas externas deben ser validadas antes de ser procesadas por las capas internas.

3.7. Comunicación HTTPS no configurada – application.properties

El archivo de configuración contiene server.port=8081 pero no contiene una configuración visible de SSL/TLS. Para una aplicación financiera que procesa información del tipo financiero, la comunicación entre clientes y servidor debe estar protegida mediante HTTPS.

4. Pruebas estáticas

El proyecto ya incluye el complemento OWASP Dependency-Check en el archivo pom.xml, usando la versión 5.3.0, así que el análisis estático debe ejecutarse sobre todas las dependencias del proyecto y hacer constar los resultados en el informe que genera Dependency-Check. En el código que se ha proporcionado se identifican de manera directa las siguientes dependencias de interés:

Summary of Vulnerable Dependencies (Click to show all)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
bcprov-jdk15on-1.46.jar	cpe:2.3:a:bouncycastle:bouncy-castle-crypto-package:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:bouncy_castle_crypto_package:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:bouncy_castle_for_java:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:legion-of-the-bouncy-castle-java-crytography-api:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:the_bouncy_castle_crypto_package_for_java:1.46:*:*:*:*:*:*:*	pkg:maven/org.bouncycastle/bcprov-jdk15on@1.46	HIGH	4	Highest	38
hibernate-validator-6.0.18.Final.jar	cpe:2.3:a:hibernate:hibernate-validator:6.0.18:*:*:*:*:*:*:* cpe:2.3:a:redhat:hibernate_validator:6.0.18:*:*:*:*:*:*:*	pkg:maven/org.hibernate.validator/hibernate-validator@6.0.18.Final	MEDIUM	3	Highest	32
jackson-databind-2.10.2.jar	cpe:2.3:a:fasterxml:jackson-core:2.10.2:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-databind:2.10.2:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-modules-java8:2.10.2:*:*:*:*:*:*:*	pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.10.2	HIGH	11	Highest	39
log4j-api-2.12.1.jar	cpe:2.3:a:apache:log4j:2.12.1:*:*:*:*:*:*:*	pkg:maven/org.apache.logging.log4j/log4j-api@2.12.1	MEDIUM	3	Highest	42
logback-core-1.2.3.jar	cpe:2.3:a:qos:logback:1.2.3:*:*:*:*:*:*:*	pkg:maven/ch.qos.logback/logback-core@1.2.3	HIGH	2	Highest	31
snakeyaml-1.25.jar	cpe:2.3:a:snakeyaml_project:snakeyaml:1.25:*:*:*:*:*:*:*	pkg:maven/org.yaml/snakeyaml@1.25	CRITICAL	8	Highest	44
spring-boot-2.2.4.RELEASE.jar	cpe:2.3:a:vmware:spring_boot:2.2.4:release:*:*:*:*:*:*	pkg:maven/org.springframework.boot/spring-boot@2.2.4.RELEASE	CRITICAL	9	Highest	39
spring-core-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*	pkg:maven/org.springframework/spring-core@5.2.3.RELEASE	CRITICAL*	29	Highest	36
spring-web-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*	pkg:maven/org.springframework/spring-web@5.2.3.RELEASE	CRITICAL*	30	Highest	34
tomcat-embed-core-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*:*:*:*:*:*	pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@9.0.30	CRITICAL*	76	Highest	30
tomcat-embed-websocket-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*:*:*:*:*:*	pkg:maven/org.apache.tomcat.embed/tomcat-embed-websocket@9.0.30	CRITICAL*	77	Highest	30

* indicates the dependency has a known exploited vulnerability

Las versiones de Spring Framework que son requeridas por Spring Boot 2.2.4 son antiguas; por ejemplo, spring-core, spring-web y otros componentes relacionados a la versión

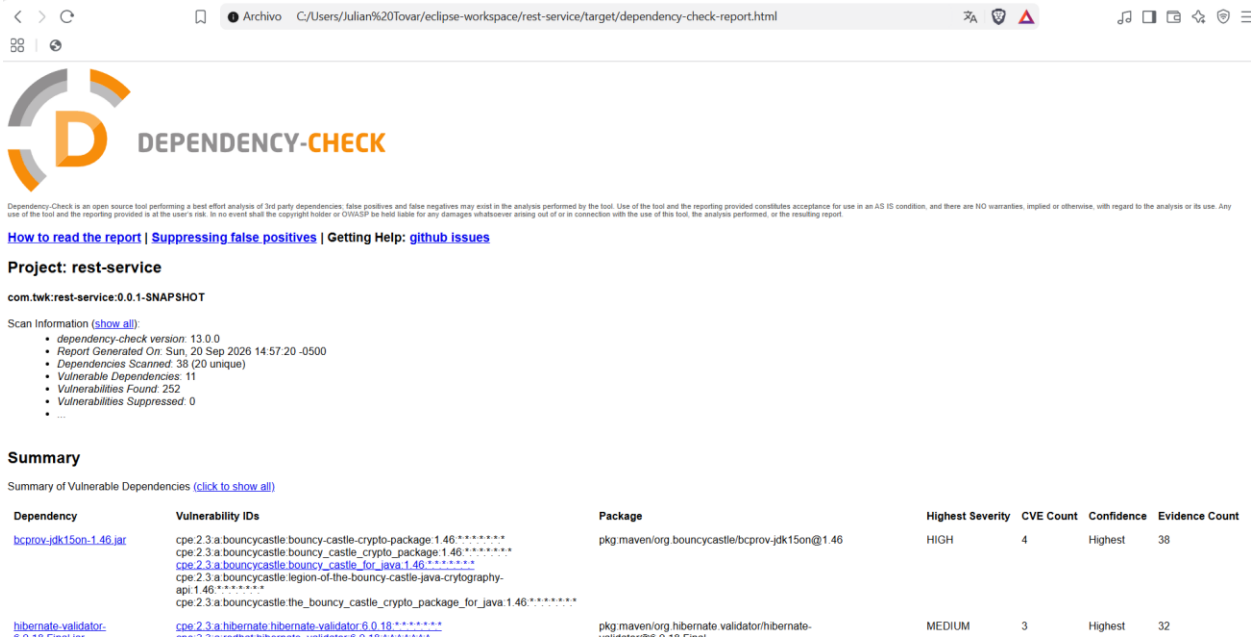
5.2.3.RELEASE. Las fuentes públicas de vulnerabilidades indican que estas versiones antiguas de Spring tienen múltiples vulnerabilidades conocidas.
Dentro de los identificadores que se identifican de manera pública asociados a estas versiones se encuentran CVE-2022-22965, CVE-2021-22118, CVE-2020-5421, CVE-2022-22950, CVE-2022-22971, CVE-2023-20861, CVE-2023-20863, CVE-2022-22968, CVE-2022-22970, CVE-2021-22060 y CVE-2021-22096, dependiendo del componente analizado. Es necesario contrastar estos identificadores con el HTML obtenido a partir de la ejecución en particular de Dependency-Check para no incluir en el árbol de dependencias falso positivo y/o vulnerabilidades que no estén identificadas con el árbol de dependencias correspondiente.
También hay que tener en cuenta a Tomcat 9.0.30 que es a su vez dependencia de Spring Boot 2.2.4. La vulnerabilidad de CVE-2019-17569 afecta a versiones más antiguas de Tomcat y se refiere a una interpretación no consistente de las solicitudes HTTP y a un potencial HTTP request smuggling en aquellas configuraciones que integran un proxy inverso.

La dependencia de la versión Bouncy Castle 1.46 es especialmente antigua. Las fuentes de la seguridad de dependencias nos avisan que hay múltiples problemas en las versiones más antiguas de bcprov-jdk15on. Por lo tanto, hay que cambiarla por una versión soportada.
Nota sobre el resultado definitivo de Dependency-Check: debe5 ejecutarse sobre el entorno de desarrollo/Codio, el cual tiene acceso a los repositorios necesarios. El resultado final se basa en el archivo dependency-check-report.html, el cual será el que determine exactamente qué CVE ha identificado el proyecto.

5. Plan de mitigación

Para remediar las vulnerabilidades identificadas se propone el siguiente plan de remediación:
Remover la utilización de credenciales codificadas en DocData.java. Reemplazar estas credenciales por variables de entorno, secrets managed o, alternativamente, un servicio de gestión de credenciales seguro.
Implantar HTTPS/TLS. Configurar un certificado y exigir que las comunicaciones a la API se realicen a través de HTTPS.
Implantar autenticación y autorización. Resguardar los endpoints con alguna solución como Spring Security y utilizar roles o permisos para gestionar el acceso a información financiera.
Validar y sanitizar cada una de entradas. Añadir validaciones para name, business_name y cualquier otro parámetro recibido por HTTP; deberían existir límites de longitud, formato y valores permitidos.
Aumentar el manejo de errores. Reemplazar printStackTrace() y utilizar un sistema de logging y evitar enviar detalles internos al cliente.
Gestionar adecuadamente las conexiones de base de datos. Utilizar try-with-resources o un mecanismo de pooling para asegurar el cierre y la gestión de las conexiones.
Reducir y controlar los límites de carga. Comprobar los valores de max-file-size y max-request-size y aceptar solo los tamaños y tipos de archivo que sean necesarios.
Validar las operaciones financieras. Modificar métodos como deposit() para prevenir valores negativos, desbordamientos e, incluso, operaciones inválidas.
Actualizar las dependencias. Comprobar y actualizar Spring Boot, Spring Framework, Tomcat, Bouncy Castle, Jackson, SnakeYAML, Logback y cualquier otra dependencia vulnerable que

Dependency-Check identifique. El objetivo debe ser el de hacer uso de versiones soportadas y compatibles entre sí.
Integrar Dependency-Check en el proceso de desarrollo. Ejecutar análisis de dependencias periódicamente y, en la medida de lo posible, incorporarlos en el proceso de integración continua para detectar nuevas vulnerabilidades antes de realizar el despliegue de los cambios.
En definitiva, la revisión lleva a la conclusión de que la aplicación tiene que integrar controles de seguridad que permiten tener un mejor control tanto del código fuente como de sus dependencias y configuración. Las medidas propuestas pueden permitir que la exposición de información financiera sea menor, que la protección de la API sea mejor y también poder llevar a cabo un mantenimiento de seguridad de forma más rutinaria.


Archivo C:/Users/Julian%20Tovar/eclipse-workspace/rest-service/target/dependency-check-report.html

DEPENDENCY-CHECK

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

How to read the report | Suppressing false positives | Getting Help: github issues

Project: rest-service

com.twk:rest-service:0.0.1-SNAPSHOT

Scan Information (show all):

- *dependency-check version*: 13.0.0
- *Report Generated On*: Sun, 20 Sep 2026 14:57:20 -0500
- *Dependencies Scanned*: 38 (20 unique)
- *Vulnerable Dependencies*: 11
- *Vulnerabilities Found*: 252
- *Vulnerabilities Suppressed*: 0
- ...

Summary

Summary of Vulnerable Dependencies (click to show all)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
bcprov-jdk15on-1.46.jar	cpe:2.3:a:bouncycastle:bouncy-castle-crypto-package:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:bouncy_castle_crypto_package:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:bouncy_castle_for_java:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:legion-of-the-bouncy-castle-java-crytography-api:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:the_bouncy_castle_crypto_package_for_java:1.46:*:*:*:*:*:*:*	pkg:maven/org.bouncycastle/bcprov-jdk15on@1.46	HIGH	4	Highest	38
hibernate-validator-	cpe:2.3:a:hibernate:hibernate-validator:6.0.18:*:*:*:*:*:*:*	pkg:maven/org.hibernate.validator/hibernate-	MEDIUM	3	Highest	32